

Sum

Объяснение решения

- Решение сделано через функцию `max_even_arr_sum`, функция получает на вход массив `numbers`, возвращает `int` - max четную сумму элементов массива
как работает функция
 - Проходимся циклом по всему массиву и считаем сумму всех элементов массива - `summ`; во время прохода запоминаем min нечетный элемент в `min_odd`
 - Если `summ` - четное число, то это уже будет max, его т.к. мы сложили все элементы массива возвращаем
 - Если `summ` - нечетное число, то чтобы сделать её четной, нужно вычесть min нечетное число из массива, оно как раз хранится в `min_odd` $\Rightarrow$ функция вернет `summ - min_odd`
так сумма уменьшится минимумом
- В обоих случаях получится max even sum*

Оценка сложности

Время - $O(n)$ 1 раз проходимся по массиву из n элементов

Память - $O(1)$ фиксированное кол-во переменных

Визуализация работы

`numbers = [9, 5, 3, 8]`

`elem` `summ` `min_odd`

- 0 None

9 9 9

5 14 5 $5 < 9$

3 17 3 $3 < 5$

8 25 3 $8 - \text{even}$
so `min_odd` doesn't change

$summ = 25$
 $25 \% 2 = 1$ $\Rightarrow$ return `summ - min_odd` = $25 - 3 = \underline{22}$